A PROJECT REPORT

Submitted by

Manpreet Singh (24BCS12215)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCEAND ENGINEERING

ChandigarhUniversity

April 2026

CERTIFICATE

This is to certify that the project report entitled “BlueHire Job Portal” is a bona fide work carried out by Manpreet Singh (24BCS12215) in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science and Engineering at Chandigarh University during the academic session 2025-2026.

The work presented in this report is based on the design and implementation of a web-based job portal that allows users to register, sign in, browse available job opportunities, apply for roles, remove applications, and review their activity through a dashboard. The project integrates a React frontend, an Express backend, and MongoDB for data persistence.

To the best of our knowledge, this report has not been submitted in part or full to any other university or institution for the award of any degree or diploma.

Project Supervisor	Student Signature

ACKNOWLEDGEMENT

I express my sincere gratitude to the faculty of the Department of Computer Science and Engineering for providing the academic environment and guidance required to complete this project successfully.

I am thankful to teachers, mentors, and lab staff whose suggestions helped in refining the structure, usability, and implementation approach of the N-Queen Solver and Visualizer.

I also acknowledge the documentation and learning resources on HTML, CSS, JavaScript, recursion, and backtracking that served as reliable technical references during development.

Finally, I thank my family and peers for their support, encouragement, and constructive discussions throughout the completion of this project.

ABSTRACT

The N-Queen Solver and Visualizer is a browser-based mini-project designed to demonstrate the classical N-Queen problem through an interactive interface. The system accepts a board size, computes all valid placements of queens using backtracking, and allows the user to move through the generated solutions one by one.

The frontend is built using HTML, CSS, and vanilla JavaScript. The application includes input validation, dynamic board creation, recursive solving logic, solution storage, and responsive rendering.

The project focuses on clarity, usability, and clean separation between interface logic and solver logic. It is appropriate as an academic semester project because it demonstrates recursion, state handling, UI rendering, and algorithm visualization in a compact but meaningful way.

TABLE OF CONTENTS

INTRODUCTION

The N-Queen problem is a well-known problem in computer science and discrete mathematics. The objective is to place N queens on an N x N chessboard in such a way that no two queens attack each other horizontally, vertically, or diagonally.

This project presents the N-Queen problem through an interactive web application. Instead of printing only one answer, the application computes all valid solutions for the selected board size and allows the user to browse them using previous and next controls.

From a software engineering perspective, the project is useful because it combines user input handling, recursive search, state management, and dynamic DOM rendering in a single compact codebase.

OBJECTIVES AND SCOPE

The main objectives of the project are as follows:

-
-To design a simple and user-friendly interface for the N-Queen problem.
-To implement the backtracking algorithm for safe queen placement.
-To generate and store all valid solutions for a selected board size.
-To allow users to navigate through multiple valid arrangements.
-To demonstrate recursion and constraint checking in a practical web-based setting.
To keep the implementation compact and suitable for academic presentation.

EXISTING SYSTEM AND PROBLEM STATEMENT

Many basic N-Queen demonstrations are static, console-based, or limited to displaying only one valid arrangement. Some implementations also omit input validation and interactive visualization.

The main problem addressed by this project is the need for a simple browser-based system where the user can choose a board size, compute solutions safely, view the board directly, move across multiple valid arrangements, and reset the interface without reloading the page.
By solving these issues, the project moves beyond a theoretical coding exercise and becomes a structured educational application.

REQUIREMENT ANALYSIS

Software requirements:

- Modern browser with HTML5, CSS3, and JavaScript support
- Local static file execution or lightweight local server
- Code editor for development and documentation work

Hardware requirements:

- Standard personal computer or laptop
- Minimum 4 GB RAM for development use
- Keyboard and pointing device for interaction

The requirements can be grouped into functional and non-functional categories.

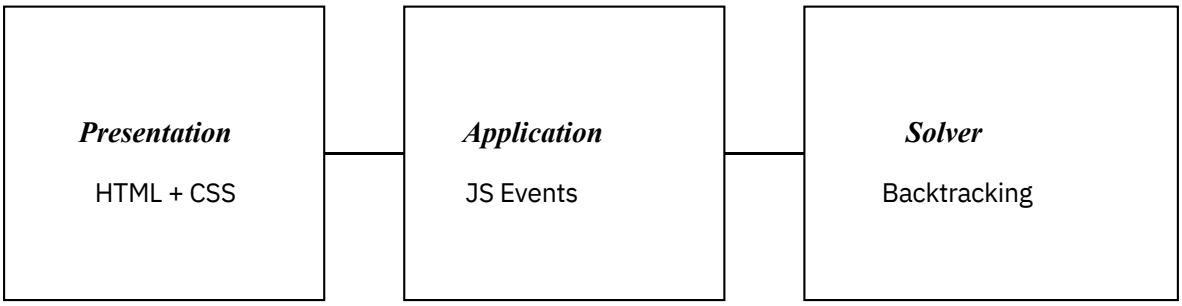
Type	*Requirement*
Functional	Accept a valid board size from 4 to 12 and compute valid queen placements.
Functional	Display the board, total solution count, and next or previous solution state.
Non-functional	Provide clear status messages and a clean, easy-to-understand interface.
Non-functional	Remain responsive for the supported input range on desktop and mobile screens.

SYSTEM ARCHITECTURE

The N-Queen project follows a lightweight three-part browser architecture. The presentation layer is handled by the HTML structure and CSS layout. The application layer is implemented through JavaScript event handlers and state variables. The solver layer performs recursive backtracking and stores valid arrangements for later display.

Component	***Responsibility***
HTML Interface	Board size input, solve and reset buttons, message area, board container, and navigation controls.
CSS Layout	Responsive container styling, chessboard grid, cell colors, and button presentation.
JavaScript Logic	State management, safety checks, recursive solving, rendering, and solutionnavigation.

Figure: High-level architecture of the N-Queen browser application



DATA REPRESENTATION

The application is frontend-only, so it does not use a database. Instead, it relies on in-memory data structures that are sufficient for the algorithm and interface workflow.

Item	***Purpose***
board	Two-dimensional array storing 0 for empty cells and 1 for queen positions.
size	Numeric value read from the board size input field.
solutions	Array containing deep copies of every valid board arrangement.
currentSolution	Index of the solution currently shown on screen.

This representation keeps the project compact while still supporting solving, rendering, and circular navigation across multiple valid outputs.

Sample board state

```
[0, 1, 0, 0]
[0, 0, 0, 1]
[1, 0, 0, 0]
[0, 0, 1, 0]
```

FUNCTION SPECIFICATION

The application exposes a compact set of JavaScript functions. Each function has a clear responsibility and contributes to a predictable solve-render-navigate cycle.

Function	***Purpose***
createBoard	Creates an N x N matrix filled with zero values.
drawBoard	Renders the current board state in the browser grid.
isSafe	Checks column, upper-left diagonal, and upper-right diagonal safety.
saveSolution	Stores a deep copy of the current valid board arrangement.
solveNQueen	Recursively explores valid queen placements row by row.
showSolution	Loads a stored solution and redraws the board.

FRONTEND IMPLEMENTATION

The frontend is implemented using plain web technologies and organized into three files. The HTML file defines the structure of the controls, board area, status message, and navigation panel. The CSS file provides the container layout, board grid, alternating cell colors, buttons, and responsive presentation. The JavaScript file manages state, validation, solving, rendering, and solution navigation.

The interface is intentionally simple so that the focus remains on the algorithm while still providing a polished user experience suitable for demonstration.

SOLVER IMPLEMENTATION

The core solver is based on recursive backtracking. The algorithm places one queen in each row. Before placing a queen, it checks whether the chosen cell is safe with
respect to the same column, the upper-left diagonal, and the upper-right diagonal.

If the position is safe, the queen is placed and the solver proceeds to the next

row.

If no safe position exists in a later row, the previous queen is removed and the next

column is tried. Every time the algorithm reaches row N, the current board is

stored as
one valid solution.
This approach is easy to explain academically and is well suited to the problem constraints enforced by the interface.

APPLICATION WORKFLOW

The operational workflow of the N-Queen application can be summarized as follows:

- The page loads and initializes an empty board using the default size.
- The user enters a value of N between 4 and 12.
- The Solve button validates the input and starts a fresh solving cycle.
- The backtracking solver computes all valid arrangements and stores them.
- The total number of solutions is displayed in the status message area.
- The first valid solution is shown on the board.
- The user navigates through other solutions using previous and next controls.
  The Reset button or input change clears the current state and redraws the board.

TESTING AND VALIDATION

The project was validated through manual interface testing and algorithm-output verification. Formal automated tests are not included in the current version, but the implemented behaviors can be validated reliably through repeatable manual cases.

- Enter 4 and click Solve: the application should display 2 valid solutions.
- Enter 8 and click Solve: the application should display 92 valid solutions.
- Enter 3 and click Solve: the system should show a validation message for the
- allowed range.
  Click Next after solving: the displayed board should move to the next
- stored
  arrangement.
- Click Previous from the first solution: navigation should wrap to the last solution.
  Click Reset after solving: the board should clear and the status should

  return to
  Ready.

RESULTS, STRENGTHS AND LIMITATIONS

The final outcome of the project is a working browser-based N-Queen solver that computes and visualizes all valid solutions for board sizes in the supported range.

The major strengths of the system are its clear demonstration of backtracking, simple and understandable interface, support for browsing all generated solutions, and responsive board rendering suitable for presentation.

The current limitations are also important to note. The application does not animate every recursive step, performance decreases for larger inputs because all solutions are generated and stored, and automated unit tests are not yet included.

CONCLUSION AND FUTURE SCOPE

The N-Queen Solver and Visualizer successfully meets the objective of implementing a classical algorithmic problem in a simple and interactive web application. The project demonstrates recursion, constraint checking, solution storage, UI rendering, and state-driven updates in a clear and academically suitable manner.

As a semester project, it provides a practical example of how theoretical problem-solving techniques can be transformed into a user-facing software tool. The future scope of the project includes animated step-by-step visualization, performance metrics, support for unique solutions under symmetry, export features,
faster safety checks using dedicated tracking arrays, and automated test coverage.

REFERENCES

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. Introduction to Algorithms.
- Donald E. Knuth. The Art of Computer Programming, Volume 4A: Combinatorial Algorithms.

-Mozilla Developer Network Web Docs for HTML, CSS, and JavaScript.

- Standard references and tutorials on recursion and backtracking.
- Project source files available in the semester_project/N_QUEEN directory submitted with this report.